

# Lesson: Fundamentals of Linked Lists

## Introduction to Linked Lists

A linked list is a data structure that consists of a sequence of elements, where each element points to the next element in the sequence. Unlike arrays, linked lists do not require contiguous memory locations, allowing for efficient memory usage and dynamic sizing.

## Basic Components of a Linked List:

- **Node:** The basic unit of a linked list that contains data and a pointer to the next node.
- **Head:** A pointer/reference to the first node in the list.
- **Tail:** A pointer/reference to the last node in the list (optional).

## Linked List Operations

Here are some fundamental operations you can perform on a singly linked list:

**1. Insertion** - Adding a new node to the linked list. This can be done at various positions:

- **At the beginning:** The new node becomes the head of the list.
- **At the end:** The new node is added after the last node.
- **At a specific position:** The new node is inserted between two existing nodes.

**2. Deletion** - Removing a node from the linked list. This can involve:

- **Deleting the first node:** The head of the list is removed, and the next node becomes the new head.
- **Deleting the last node:** The last node is removed, and the second-to-last node's next pointer is set to null.
- **Deleting a specific node:** A node is removed based on its value or position, updating the pointers of adjacent nodes accordingly.

**3. Traversal** - Visiting each node in the linked list, usually to read or process the data stored in each node. This is typically done starting from the head and moving through each node using the next pointers until the end of the list is reached.

**4. Search** - Finding a node in the linked list that contains a specific value. This operation involves traversing the list and checking each node's value until the desired value is found or the end of the list is reached.

These operations are fundamental for manipulating and utilizing linked lists in various applications.

### Sample C++ Code

```
#include <iostream> // Include the iostream library for input and output

using namespace std; // Use the standard namespace to avoid prefixing std::

// Define a structure for the nodes of the linked list
struct Node {
    int data; // Data part of the node
    Node* next; // Pointer to the next node

    // Constructor to initialize a new node
    Node(int value) {
        data = value; // Set the data
        next = nullptr; // Initialize next pointer to nullptr
    }
};

// Define a class for the linked list
class LinkedList {
private:
    Node* head; // Pointer to the head (first node) of the list

public:
    // Constructor to initialize an empty linked list
    LinkedList() {
        head = nullptr; // Set head to nullptr (list is empty)
    }

    // Function to insert a new node at the end of the list
    void insert(int value) {
        Node* newNode = new Node(value); // Create a new node with the given value
        if (head == nullptr) { // If the list is empty
            head = newNode; // Set head to the new node
        } else {
            Node* temp = head; // Start from the head
            while (temp->next != nullptr) { // Traverse to the last node
                temp = temp->next; // Move to the next node
            }
            temp->next = newNode; // Link the last node to the new node
        }
    }

    // Function to display the list
    void display() {
        Node* temp = head; // Start from the head
```

```

        while (temp != nullptr) { // While there are nodes to display
            cout << temp->data << " "; // Print the data of the current node
            temp = temp->next; // Move to the next node
        }
        cout << endl; // Print a newline at the end
    }

    // Destructor to clean up the linked list
    ~LinkedList() {
        Node* temp;
        while (head != nullptr) { // While the list is not empty
            temp = head; // Store the current head
            head = head->next; // Move head to the next node
            delete temp; // Delete the old head node
        }
    }
};

int main() {
    LinkedList list; // Create an instance of the linked list
    list.insert(10); // Insert 10 into the list
    list.insert(20); // Insert 20 into the list
    list.insert(30); // Insert 30 into the list

    cout << "Linked List: "; // Output prompt
    list.display(); // Display the linked list

    return 0; // Indicate successful termination
}

```

## Explanation

1. **#include <iostream>**: This includes the standard input-output stream library, allowing us to use cout for output.
2. **using namespace std;**: This line allows us to use names from the std namespace without prefixing them with std::.
3. **struct Node**: Defines a structure for a node in the linked list. Each node contains:
  - **int data**: To store the value of the node.
  - **Node\* next**: A pointer to the next node in the list.
4. **Node(int value)**: Constructor for Node, initializes data with the given value and next with nullptr.
5. **class LinkedList**: Defines the LinkedList class, which will manage the linked list.
6. **Node\* head;**: A private pointer to the first node of the linked list.
7. **LinkedList()**: Constructor for the LinkedList class, initializes head to nullptr, indicating an empty list.

8. **void insert(int value):** A method to add a new node with a given value to the end of the list:
  - Creates a new node.
  - If the list is empty, it sets head to the new node.
  - Otherwise, it traverses to the end of the list and links the new node.
9. **void display():** A method to print all the values in the list:
  - Starts from head and iterates through each node, printing the data.
10. **~LinkedList():** Destructor for the linked list to free allocated memory:
  - Iterates through the list, deleting each node until the list is empty.
11. **int main():** The main function:
  - Creates a LinkedList object.
  - Inserts some values into the list.
  - Call the display method to print the list.

## Conclusion

Linked lists are fundamental data structures that can be used to implement various abstract data types. Understanding linked lists is crucial for further studies in data structures and algorithms. In this lesson, we've covered the basics of linked lists and provided a sample implementation in C++. Practice implementing and modifying the code to deepen your understanding!